Lab 3 – Data Processing I

From file to tidy table

Course text for the notebook `lab3_data_processing_I.py`

dr. Maarten Slembrouck

What is this lab about?

In the previous two labs you learned the basics of Python: variables, functions, lists, dictionaries and loops. At the end of lab 2 you read `athletes.csv` with the `csv` module and used a `for` loop to count the athletes per position. That worked, but it took a lot of code. From this lab on you use **pandas**, a library built for exactly this kind of work.

The story stays the same: you are the new data analyst of a youth football and athletics club. The head coach gives you three export files. They are messy on purpose, the way real exports are:

- `athletes.csv`: one row per athlete (ID, name, birthdate, gender, position, height, weight);
- `tests.csv`: results of physical tests (10 m and 30 m sprint, jump height, Yo-Yo level) at three test moments;
- `hr_session.csv`: heart rate and speed per second during one training session (you only use it in lab 4).

After this lab you can load and explore a dataset, select and filter rows and columns, compute new columns, and clean the data until you can trust it.

How to use this text. Read it next to the notebook. Every code example below was actually run on the club's data; the *output* box shows what Python returned. Type the examples into an empty cell yourself: you learn more from code you type than from code you read. The exercises are in the notebook; this text explains what you need to solve them.

1 A few Python concepts you need today

pandas is ordinary Python. Four things from the previous labs keep coming back:

- **Calling a function** uses parentheses: `round(5.3456, 2)`. The values between the parentheses are called *arguments*.
- **Named arguments.** Some arguments are passed with their name: `round(number=5.3456, ndigits=2)`. You see this a lot in pandas, e.g. `ascending=False`.
- **A method** is a function that belongs to an object. You call it with a dot: `"Forward".lower()` gives `"forward"`. A pandas table has dozens of methods.
- **Lists and dictionaries**: `["name", "position"]` is a list, `{"fwd": "Forward"}` is a dictionary that links a key to a value.

2 What is pandas?

pandas is a **library**: a collection of code that someone else wrote and that you can use in your own program. You load it with `import`. Everyone uses the short name `pd`:

```
import pandas as pd
```

pandas revolves around two kinds of objects:

- a **DataFrame** is a table with rows and columns, like a worksheet in Excel;
- a **Series** is one column of such a table.

Every row has a label, the **index**. By default that is simply a number, starting at 0. You can make a small DataFrame yourself from a dictionary: every key becomes a column name, every list the values of that column.

```
mini = pd.DataFrame({
    "name": ["Yana", "Stan", "Emma"],
    "position": ["Defender", "Midfielder", "Forward"],
    "height_cm": [167.1, 158.1, 171.0],
})
mini
```

output

```
   name    position  height_cm
0  Yana    Defender      167.1
1  Stan  Midfielder      158.1
2  Emma     Forward      171.0
```

The column on the left (0, 1, 2) is the index; it is not a real column of data. Asking for one column gives a Series:

```
mini["height_cm"]
```

output

```
0    167.1
1    158.1
2    171.0
Name: height_cm, dtype: float64
```

Key idea

A DataFrame is a table, a Series is one column. Almost everything you do today is: take a DataFrame, do something with it, and get a new DataFrame or Series back.

The big advantage over lists: a calculation on a Series works on *every* row at once, without a `for` loop.

```
mini["height_cm"] / 100
```

output

```
0    1.671
1    1.581
2    1.710
Name: height_cm, dtype: float64
```

3 Loading a CSV file

A CSV file (*comma-separated values*) is plain text: one line per row, with commas between the values. The first line contains the column names. This is what the start of `tests.csv` looks like as text:

output
```
test_id,athlete_id,test_moment,date,sprint_10m_s,sprint_30m_s,cmj_height_cm,yoyo_level
1,A001,1,2026-09-14,2.03,5.34,20.6,19.0
2,A001,2,2026-10-19,2040.0,5.32,21.2,20.0
3,A002,1,2026-09-14,2360.0,5.97,30.4,14.0
```

To Python this is just text: it does not know that `5.34` is a number, or that `date` contains a date. `pd.read_csv` reads the file and turns it into a DataFrame, with a suitable data type per column:

```
athletes_raw = pd.read_csv(f"{DATA_DIR}/athletes.csv")
athletes_raw.head()
```

output

	athlete_id	name	birthdate	gender	position	height_cm	weight_kg
0	A001	Yana De Backer	2010-05-27	F	defender	167.1	65,5
1	A002	Stan Mertens	2012-02-19	M	midfielder	158.1	59,6
2	A003	Amber Jacobs	2010-08-19	F	Defender	168.0	63,9
3	A004	Bram Wouters	2009-04-07	M	MID	177.8	68,7
4	A005	Emma Verhoeven	2010-10-18	F	goalkeeper	170.1	61,1

`DATA_DIR` is a variable created at the top of the notebook: the folder with the data. Locally that is the `data/` folder, online an address on GitHub. With `f"{DATA_DIR}/athletes.csv"` (an *f-string*) you add the file name to it.

Watch out: names in marimo

In marimo, two cells cannot create the same variable. So give every step its own name: `athletes_raw` for the raw data, `athletes_clean` for the cleaned version. That also makes your code easier to read.

4 A first impression of the data

Before you do anything with a dataset, you look at it. Five tools give you a quick overview.

`.head()` shows the first five rows (or `.head(10)` for ten). You saw that above.

`.shape` gives the number of rows and columns. Note: there are no parentheses after it, because `shape` is a property, not a method.

```
athletes_raw.shape
```

output
```
(43, 7)
```

The result is a *tuple*: a fixed sequence of values between parentheses. `athletes_raw.shape[0]` is the number of rows (43), `athletes_raw.shape[1]` the number of columns (7).

`.dtypes` shows which data type pandas chose for each column:

```
athletes_raw.dtypes
```

output
```
athlete_id        str
name              str
birthdate         str
gender            str
position          str
height_cm     float64
weight_kg         str
dtype: object
```

`int64` is a whole number, `float64` a decimal number and `str` text. Notice anything? `weight_kg` is text, while a weight should be a number. That is the first problem you solve today. `birthdate` is text too; pandas does not recognise a date by itself.

`.info()` combines that with the number of filled-in (*non-null*) values per column. `.describe()` gives statistics for the numeric columns: count, mean, standard deviation (`std`), minimum, quartiles and maximum.

```
athletes_raw.describe()
```

output

```
        height_cm
count   41.000000
mean   163.712195
std      9.465891
min    141.100000
25%    157.100000
50%    163.900000
75%    170.100000
max    179.200000
```

`count` is 41 for `height_cm`, while there are 43 athletes: two athletes have no height. A table like this is a quick way to spot missing and impossible values.

`.value_counts()` counts how often each value occurs in a column. What you did in lab 2 with a loop and a dictionary is now a single line:

```
athletes_raw["position"].value_counts()
```

output

```
position
FWD           6
defender      5
forward       5
midfielder    4
MID           4
goalkeeper    4
Midfielder    4
Defender      3
defender      3
Forward       2
GK            1
forward       1
DEF           1
Name: count, dtype: int64
```

The club has only four positions, but the column contains 13 different spellings. That is the second problem of today.

5 Selecting columns

You select one column with square brackets and the column name, and get a Series back. You select several columns with a *list* of column names between the brackets, and get a DataFrame back.

```
athletes_raw[["name", "position"]].head(3)
```

output

```
             name    position
0  Yana De Backer   defender
1    Stan Mertens  midfielder
2    Amber Jacobs    Defender
```

Watch out: double brackets

The outer brackets mean "select", the inner ones make a list. `athletes_raw["name", "position"]` (with single brackets) gives a `KeyError`: pandas then looks for one column with the strange name `("name", "position")`.

6 Filtering rows

Filtering takes two steps. First you write a **condition**. A comparison on a Series gives a new Series with `True` or `False` for every row:

```
is_tall = athletes_raw["height_cm"] > 175
is_tall.head()
```

output

```
0    False
1    False
2    False
3     True
4    False
Name: height_cm, dtype: bool
```

Such a Series of `True`/`False` is called a **mask**. Then you pass the mask to `.loc[...]`. You keep only the rows where the mask is `True`:

```
athletes_raw.loc[is_tall, ["name", "position", "height_cm"]]
```

output

```
                name    position  height_cm
3       Bram Wouters         MID      177.8
6     Lotte Janssens          GK      176.3
9   Mathis Verhoeven   defender       179.2
17     Julie De Smet  goalkeeper      177.0
25   Amber Verhoeven     forward      178.8
```

So `.loc[rows, columns]` selects rows *and* columns in one go. Leave out the second part and you get all columns.

6.1 Filtering on text

You compare text with `==`. The comparison is exact: upper case and spaces count.

```
only_forwards_raw = athletes_raw.loc[athletes_raw["position"] == "Forward"]
only_forwards_raw.shape
```

output

```
(2, 7)
```

Only 2 rows, while `value_counts()` also showed `"forward"`, `"forward "` and `"FWD"`. Filtering on text only works well once the labels are cleaned (section 8).

6.2 Filtering on a date

`birthdate` is text. `pd.to_datetime` turns it into real dates, which you can compare. Anyone born after 1 September 2010 is younger than 16 in the 2026 season:

```
younger_than_16 = pd.to_datetime(athletes_raw["birthdate"]) > "2010-09-01"
younger_than_16.sum()
```

output

```
28
```

`.sum()` on a mask counts the `True` values, because Python counts `True` as 1 and `False` as 0. So 28 athletes are younger than 16.

6.3 Combining conditions

You combine two conditions with `&` (and), `|` (or) and `~` (not). Put every condition between parentheses:

```
tall_girls = athletes_raw.loc[
    (athletes_raw["gender"] == "F") & (athletes_raw["height_cm"] > 170)
]
tall_girls[["name", "gender", "height_cm"]]
```

output

```
               name gender  height_cm
4    Emma Verhoeven      F      170.1
6    Lotte Janssens      F      176.3
17    Julie De Smet      F      177.0
21    Hanne Willems      F      172.8
24     Roos Lemmens      F      171.3
25  Amber Verhoeven      F      178.8
39   Hanne Michiels      F      173.3
```

Watch out: `and` does not work here

In plain Python you write `and` and `or`. On a Series that does not work: you get a `ValueError` (*The truth value of a Series is ambiguous*). Use `&` and `|`, and don't forget the parentheses around each condition. Without them Python evaluates `&` first, and the condition is no longer what you meant.

6.4 Sorting

`.sort_values("column")` sorts the rows from small to large. With `ascending=False` you reverse the order. The index moves along, so you can still see which row each value came from.

```
tests_raw = pd.read_csv(f"{DATA_DIR}/tests.csv")
slowest_10m = tests_raw.sort_values("sprint_10m_s", ascending=False)
slowest_10m[["athlete_id", "sprint_10m_s"]].head()
```

output

```
    athlete_id  sprint_10m_s
2         A002        2360.0
67        A027        2220.0
106       A041        2160.0
83        A033        2120.0
7         A004        2100.0
```

A 10 m sprint of 2360 seconds? Those values are stored in *milliseconds* instead of seconds (2.36 s

became 2360). That is the third problem: the same mistake is in `sprint_30m_s`.

Try it yourself

How many athletes are taller than 180 cm? Use a mask and `.sum()`. Then sort `athletes_raw` from tall to short and look at the first five rows.

7 Computing new columns

Making a new column is an assignment: `df["new_column"] = ...`. The right-hand side is usually a calculation on other columns, done for all rows at once.

```
speed = tests_raw[["athlete_id", "sprint_30m_s"]].copy()
speed["speed_kmh"] = 30 / speed["sprint_30m_s"] * 3.6
speed.head(3)
```

output

```
  athlete_id  sprint_30m_s  speed_kmh
0       A001          5.34  20.224719
1       A001          5.32  20.300752
2       A002          5.97  18.090452
```

30 metres divided by the time gives metres per second; times 3.6 gives kilometres per hour. Note that `30 / speed["sprint_30m_s"]` divides a number by a whole column: pandas does the division for every row.

Why `.copy()`?

`speed = tests_raw[...]` sometimes does not make a new table but a *view* on the old one. If you then add a column, you can't be sure whether the original table changes too. `.copy()` makes a real, separate copy. That is why the notebook exercises start with `athletes_raw.copy()`: the raw data stays untouched.

7.1 Turning text into a number

`weight_kg` contains text like `"57,5"`, with a comma as the decimal separator. That often happens in exports from European software. Python expects a dot. You fix it in two steps:

```
weight_kg_numeric = athletes_raw["weight_kg"].str.replace(",", ".").astype(float)
weight_kg_numeric.head(3)
```

output

```
0    65.5
1    59.6
2    63.9
Name: weight_kg, dtype: float64
```

- `.str` gives access to text methods for *every* row; `.str.replace(",", ".")` replaces the comma with a dot.
- `.astype(float)` turns the text into a decimal number. Try it without replacing the comma first and you get a `ValueError`: `"57,5"` is not a number to Python.

You can put methods one after the other like this (*chaining*): each method works on the result of the previous one.

7.2 Calculating with dates

For the age, you subtract the birthdate from a fixed reference date. A fixed date (and not "today") makes sure your results are still the same tomorrow.

```
REFERENCE_DATE = pd.Timestamp("2026-09-14")
birthdates = pd.to_datetime(athletes_raw["birthdate"])
(REFERENCE_DATE - birthdates).head(3)
```

output

```
0   5954 days
1   5321 days
2   5870 days
Name: birthdate, dtype: timedelta64[us]
```

The difference between two dates is a *duration* (a `Timedelta`). `.dt.days` takes out the number of days, and `// 365` divides by 365 and keeps only the whole part:

```
age_years = (REFERENCE_DATE - birthdates).dt.days // 365
age_years.head(3)
```

output

```
0    16
1    14
2    16
Name: birthdate, dtype: int64
```

`//` is *integer division*: `5954 // 365` is 16, not 16.31. That gives an age in whole years, as you are used to.

8 Cleaning the data

Cleaning means: solving the problems you found while exploring, one by one, and being able to explain every choice.

8.1 Missing values

An empty cell in a CSV file becomes `NaN` (*not a number*) in pandas. `.isna()` gives a mask that is `True` where a value is missing; `.isna().sum()` counts them per column:

```
tests_raw.isna().sum()
```

output

```
test_id          0
athlete_id       0
test_moment      0
date             0
sprint_10m_s     0
sprint_30m_s     6
cmj_height_cm    6
yoyo_level       6
dtype: int64
```

You have two choices:

- `.dropna()` throws away the rows with a missing value. Reasonable when few rows are affected and you can't estimate the value well (e.g. a missing gender).
- `.fillna(value)` fills in the gaps, e.g. with the mean of the column. Reasonable when you

have a good estimate and need the row for your analysis.

Key idea

There is no rule that is always right. Decide per column and write down *why*. "Fill everything with 0" is almost always wrong: a missing jump height is not the same as a jump of 0 cm.

8.2 Cleaning text labels

The positions appear in 13 ways in the data. Two text methods solve most of it: `.str.strip()` removes spaces at the start and end, `.str.lower()` makes everything lower case.

```
position_normalized = athletes_raw["position"].str.strip().str.lower()
position_normalized.value_counts()
```

output

```
position
defender      11
midfielder     8
forward        8
fwd            6
mid            4
goalkeeper     4
gk             1
def            1
Name: count, dtype: int64
```

From 13 to 8 variants. What remains are abbreviations. You translate them with a dictionary and `.map(...)`: every value is looked up as a key and replaced by the matching value.

```
POSITION_MAP = {
    "forward": "Forward", "fwd": "Forward",
    "midfielder": "Midfielder", "mid": "Midfielder",
    "defender": "Defender", "def": "Defender",
    "goalkeeper": "Goalkeeper", "gk": "Goalkeeper",
}
position_clean = position_normalized.map(POSITION_MAP)
position_clean.value_counts()
```

output

```
position
Forward       14
Defender      12
Midfielder    12
Goalkeeper     5
Name: count, dtype: int64
```

Watch out

A value that is not in the dictionary is replaced by `NaN` by `.map`. So always check with `.value_counts()` or `.isna().sum()` that nothing got lost.

To put the cleaned column in the table, you overwrite the old column with an assignment, on a copy of the raw table.

8.3 Numbers in the wrong unit

No human runs 10 or 30 metres in under 1 second, and no sprint takes 20 seconds. So a value above 20 is in milliseconds and must be divided by 1000. You do that with `.where(condition, other)`: where the condition is `True`, the value stays; where it is `False`, the value of `other` takes its place.

```
def fix_sprint_units(seconds_column):
    return seconds_column.where(seconds_column < 20, seconds_column / 1000)

sprint_10m_fixed = fix_sprint_units(tests_raw["sprint_10m_s"])
sprint_10m_fixed.describe()
```

output

```
count    114.000000
mean       2.099912
std        0.149415
min        1.730000
25%        1.975000
50%        2.090000
75%        2.200000
max        2.470000
Name: sprint_10m_s, dtype: float64
```

The maximum is now 2.47 seconds: realistic. Because you need the same step on two columns, it lives in a **function**. That way you write the rule once and use it as often as you need.

8.4 Duplicates

`.duplicated(subset=[...])` gives `True` for every row that has the same values in the given columns as a row *above* it. `.drop_duplicates(subset=[...], keep="first")` throws those rows away and keeps the first one. A small example:

```
sessions = pd.DataFrame({
    "name": ["Yana", "Stan", "Yana"],
    "day": ["mon", "mon", "mon"],
    "minutes": [60, 45, 60],
})
sessions.duplicated(subset=["name", "day"])
```

output

```
0    False
1    False
2     True
dtype: bool
```

In `athletes.csv` one athlete appears twice, with the same name and birthdate but two different IDs: a data entry mistake. That is why you look for duplicates by *name and birthdate*, not by ID.

8.5 All together: one pipeline

At the end of the lab, the notebook puts all steps one after the other in a single block of code. Such a sequence of steps is called a **pipeline**. You meet two new things there:

- `.assign(column=...)` creates (or overwrites) a column and returns a new DataFrame. So you can put another method after it, which you can't do with `df["column"] = ...`.
- `lambda d: ...` is a short function without a name. In `.assign(bmi=lambda d:`

`d["weight_kg"] / ...`), `d` stands for the table *as it is at that point in the pipeline*. That lets you use a column made one step earlier.

```
athletes_clean = (
    athletes_raw.assign(
        position=position_clean,
        weight_kg=weight_kg_numeric,
        age=age_years,
    )
    .assign(bmi=lambda d: d["weight_kg"] / (d["height_cm"] / 100) ** 2)
    .drop_duplicates(subset=["name", "birthdate"], keep="first")
    .reset_index(drop=True)
)
```

```
athletes_clean[["name", "position", "weight_kg", "age", "bmi"]].head(3)
```

output

	name	position	weight_kg	age	bmi
0	Yana De Backer	Defender	65.5	16	23.457861
1	Stan Mertens	Midfielder	59.6	14	23.844187
2	Amber Jacobs	Defender	63.9	16	22.640306

The parentheses around the whole pipeline let you put every step on its own line. After removing a duplicate, one row number is missing; the last step, `.reset_index(drop=True)`, numbers the rows again from 0.

9 Looking ahead: the challenge

The challenge at the end of the notebook asks for the 5 fastest athletes per position. For that you need to combine two tables (`athletes_clean` has the position, `tests_clean` the sprint times) and rank per group. The hints in the notebook show the three functions you need: `groupby`, `merge` and `.head(5)` per group. Lab 4 explains them in depth; try the challenge with the hints now, and see next week whether you understand it better.

10 Summary

Code	What it does
`pd.read_csv(path)`	read a CSV file as a DataFrame
`df.head()`, `df.shape`, `df.dtypes`	first rows, (rows, columns), data types
`df.info()`, `df.describe()`	overview, statistics of numeric columns
`df["col"].value_counts()`	how often each value occurs
`df["col"]`, `df[["a", "b"]]`	one column (Series), several columns (DataFrame)
`df.loc[mask, columns]`	filter rows and choose columns
`(cond1) & (cond2)`, `|`, `~`	combine conditions: and, or, not
`df.sort_values("col", ascending=False)`	sort, from large to small
`df["new"] = ...`, `df.assign(...)`	create a new column
`.str.replace()`, `.str.strip()`, `.str.lower()`	text operations on every row
`.astype(float)`	convert the data type
`pd.to_datetime()`, `.dt.days`	text to date, number of days in a duration
`.isna().sum()`, `.dropna()`, `.fillna()`	count, drop, fill missing values
`.map(dictionary)`	translate values with a dictionary
`.where(condition, other)`	replace values where the condition is `False`
`.duplicated()`, `.drop_duplicates()`	find, remove duplicates

Common mistakes

- `KeyError`: a column name does not exist. Check the spelling with `df.columns`.
- `and`/`or` instead of `&`/`|`, or no parentheses around each condition.
- Changing a column without making a `.copy()` of the table.
- Forgetting that `==` compares text exactly (spaces, upper case).
- Using the same variable name in two cells in marimo.

Glossary

DataFrame table with rows and columns.

Series one column of a DataFrame.

index the labels of the rows (by default 0, 1, 2, . . .).

data type kind of value in a column: `int64`, `float64`, `str`, date.

mask Series of `True`/`False`, used to filter rows.

NaN missing value.

pipeline sequence of operations applied to the data one after the other.